Studio: from a one-line objective to a checked launch pack

A web app where AI agents take a marketing campaign from a one-line objective to on-brand ads in every size and language, each one checked against the brand's rules, with a person approving every step that spends money, publishes, or replaces work.

Rory McClelland · mcclelland.info

Sprint 8 of 11 · November 2026 · walkthrough at mcclelland.info/work/studio

37 s

from a signed-off brief to 12 ads on production in Sprint 5, for about 14.7p in model costs at 1K pictures

64 ads

in one launch pack: 4 picked ads in the 4 social sizes and 4 languages, built in 19.7 seconds in CI in Sprint 7

10 rules

checked on every ad in every size and language, as pass, review, or fail, with the reason and a fix

8 sprints

in 11 days, from the first decision on 24 October to Sprint 8's sign-off on 3 November

Marketing teams that run paid campaigns repeat the same production work for every one of them. A brief is pulled together from reports, research, and past campaigns. Copy and pictures are made, then every ad is resized for each channel, translated, and checked against brand, legal, and accessibility rules before anything goes live. Much of that work is checking, and it is where mistakes slip through: a legal line missing from one language, a banned phrase, words a platform will crop, or text too faint to read.

I built Studio to show how a team of AI agents can take on that production work while people keep the decisions that matter: what the campaign is for, which brief is right, which ads go, and whether a judgement call is acceptable. I owned the product: I set the sprint themes, answered the questions each refinement put to me, and signed every sprint off. Claude, as lead engineer, wrote the refinements, the code, the tests, and the documentation, and kept the backlog, under a working agreement written down on 2 November, in Sprint 7. Studio runs on the public web for invited users, with two fictional brands and simulated customer data. Its Performance squad, which will watch how creative does and propose replacements for a person to approve, comes in a later sprint.

Studio is a demonstration with no commercial aim: it is not for sale, and it is not offered as a service. The idea was inspired by multi-agent campaign platforms I saw while working at Elsewhen. I built Studio independently, after I left, and it uses none of Elsewhen's code, data, or designs.

01

Agents propose. People decide. Code checks everything in between.

What it does

Studio covers a campaign from the objective to the files a team hands to its media buyers. Each brand has a kit in a shared Business Context store (its voice, palette, fonts, legal lines, banned terms, claims that need evidence, and do and don't rules), and every agent and every check works from it.

STEP	USUALLY DONE BY HAND	WITH STUDIO
Market context	Searched for and pasted into a deck	The Market agent searches the category and writes dated notes, each linked to the pages it read; code checks every figure against those pages
Brief	Assembled from reports and past campaigns by whoever has time	The Brief agent writes it from the brand's context, citing each source; code refuses an unsupported figure or claim, and a person signs a version off
Strategy check	Rarely done before money is spent	Synthetic personas score the brief at sign-off, and a test plan sets out what to compare; both are labelled as synthetic
Copy and pictures	Briefed to a writer and a designer, then reviewed	Copy and Imagery agents work in parallel; each ad is composed in code from the picture, the words, the logo, and the legal lines
Sizes and languages	Each one resized and translated by hand	Seven sizes drawn in code; the Localise agent translates the words, and each language's legal lines come from the brand's kit, never from translation
Compliance	A checklist, applied ad by ad when there is time	Ten rules on every file, with the reason and a fix; a person settles the review items, and an agency's own ad can be checked against the same kit
Hand-off	Files collected, renamed, and sent	One launch pack with manifests, holding only the files that passed or that a person accepted

02

Measured against Studio's own targets, not yet against a team working by hand.

Faster, cheaper, fewer errors, and repeatable

Each aim is built into a different part of the design, and the speed and cost figures are measured on production or in CI.

Faster

- A signed-off brief became 12 checked ads in 37 seconds on production, against a target of about two minutes
- On production: a brief in 39 seconds, market notes in 37, audiences in 11
- Every size and language of an ad is drawn and checked at once
- People's time goes on the review items, not on the passes

Cheaper

- About 14.7p in model costs for 12 ads at 1K pictures; about 20p is the current estimate at 2K
- The typical and worst-case cost are shown on every creative, translation, and checking button before the click
- A daily cap is checked before every model call, with a separate limit for each invited workspace
- The hosting runs on free plans; only AI usage varies

Fewer errors

- Legal lines are copied by code in every language, never written by a model
- The image model never draws text, so words, logos, and legal lines are drawn exactly as written
- Figures must appear in their cited source, and real company names and testimonials are refused
- A model's "no issues" is never shown as a pass

Repeatable

- Ads are drawn the same way every time, with no network calls, and the square ad is checked pixel for pixel across releases
- One kit per brand, read by every agent and every check
- Every run is recorded with its model, tokens, cost, and time
- Thresholds sit in one configuration file, stated as Studio's own

The time and cost figures are measured against the targets set for each sprint, such as about two minutes and under 20p for a round of creative. A real deployment would start by recording a baseline: the time from objective to approved creative, the hours spent resizing, translating, and checking, and the share of ads changed in review. The saving would then be measured against it.

03

Every model output passes through code or a person before it can leave.

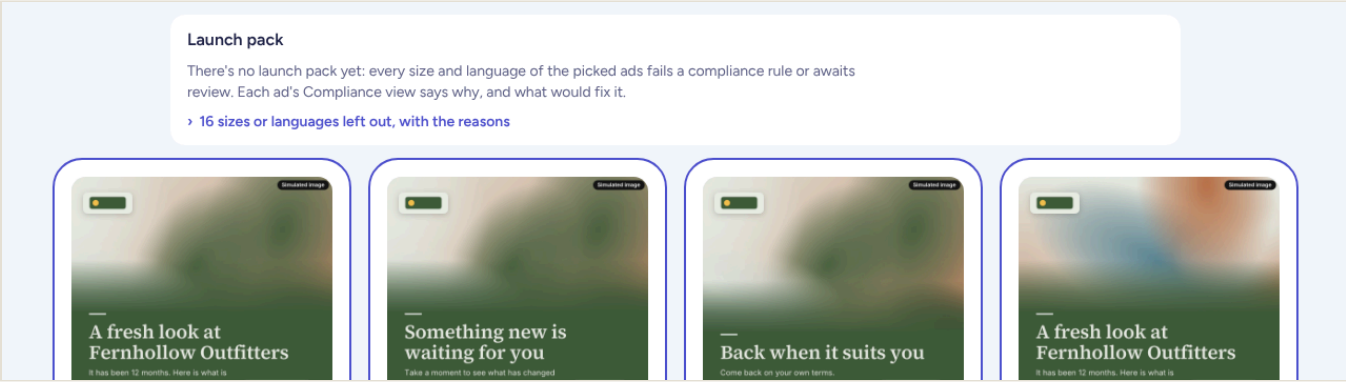
How the AI works

Studio has eleven product agents in two squads, Strategy and Creative, and an image model. Models are used where judgement about language or pictures is needed. Everything that must be exact (the layout, the legal lines, the checks, and the costs) is ordinary code, which behaves the same way every time and can be tested exhaustively.

STAGE	A MODEL SUGGESTS	CODE DECIDES	A PERSON DECIDES
Context	**Market agent** (Claude Sonnet, web search) writes dated notes on the category	Reads every cited page and drops a note whose figures are not on it; masks real company names before the brief sees them	Refreshes the notes when needed
Brief	**Brief agent** (Sonnet) writes a brief citing the brand's context items; **Audience agent** (Haiku) names the segments code has counted	Refuses a citation outside the context, a figure not in its source, a banned term, an unevidenced claim, or a real company name	Revises, then signs off one version
Strategy check	**Personas** and **test plan** (Haiku) score the brief and set out what to compare	Labels both as synthetic and keeps figures out of the test plan	Decides whether to revise
Creative	**Copy** (Sonnet) writes the words; **Imagery** (Haiku) writes picture ideas; the **image model** (Gemini) draws the picture only	Composes each ad: layout, words, logo, legal lines, and fonts, with contrast worked out before drawing; refuses testimonials and loose quantities	Picks, discards, or edits the words
Sizes and languages	**Localise** (Sonnet) translates the words, badged as machine translation	Draws seven sizes and copies each language's own legal lines, from the kit, into every one	Keeps the kit's legal lines for each language
Compliance	**Picture check** (Haiku, vision) flags faces for finance brands and possible breaches of don't rules	Ten rules on every file: pass, review, or fail, with the reason and a fix	Accepts or rejects each review item
Hand-off	*No model*	Builds one zip with JSON and CSV manifests and a README, holding only what passed or was accepted	Downloads it; Studio publishes nothing

Under every model call: the worst-case cost is reserved against the daily cap first, the answer must match a typed schema (one repair attempt, then the run stops), and the step is recorded with its inputs, output, model, tokens, cost, time, and outcome.

One campaign, from objective to launch pack. Model names are those configured for production; the brands in the app are fictional.



A campaign's gallery, with the free mock model the automated tests use, so the pictures are labelled as simulated. Four ads are picked and translated, the panel has scored them, and the launch pack waits until a person has settled the three don't rules on each ad.

04

The full set of decisions is written up as 25 numbered records.

How it is built, deployed, and run

Studio runs on a handful of hosted services, with the app's functions and its database both in London, and none of them needed custom infrastructure.

PART	CHOICE	WHY IT MATTERS
Web app	Next.js 16 on Vercel, functions in London	Screens, sign-in, the spend cap, the checks, and ad drawing; every model call goes through one gate in this code
Database	Postgres on Neon (London), with row-level security	Every business table carries a workspace, and the database itself refuses another workspace's rows
Workflows	Inngest	Each agent call is a retryable step, and each approval is a pause that waits for a person
Models	Claude Sonnet and Haiku, and a Gemini image model, behind one provider layer	Each agent's model is a configuration change, and a free mock model runs the tests
Sign-in	Studio's own emailed one-time links, sent by Resend	No passwords; links work once for 15 minutes, and only a hash of each session token is stored

Agents as small, typed definitions

Each agent is a definition: what it receives, what it must return (as a schema), its instructions, and an offline mock answer. The framework around it reserves the cost, makes the call, validates the answer, tries one repair, and records the trail, so a new agent inherits every control. Spend is reserved before each call, under a database lock, at the worst case the call could cost, so parallel calls cannot overshoot the cap. The demo's cap is £1 a day: 50p is held back for me, and each invited workspace may use up to 50p of the rest.

From a change to the live site

1. **Refine.** Stories, agreed test cases, risks, and questions with recommendations, before any code. Building starts when I say go.
2. **Test first, then build.** Every agreed case says how it will be checked: a browser test, another automated check, or by hand, with the reason.
3. **CI on every push.** Secret scan, types, lint, unit and database tests, mock evaluations, a dependency audit, and a build; recorded browser tests against the app; and the migrations applied to a throwaway Neon branch, with the database tests run there.
4. **Independent QA and review.** A QA agent that did not build the work checks the tests against the agreed cases; a reviewer agent checks security and correctness.
5. **Sign-off and deploy approval.** I sign off from a demo page built from CI's results and recordings, then approve the deploy separately.
6. **Migrations, then merge.** Database changes are tried on a fresh Neon branch first, then applied to production and preview, each checked against a fresh build by a fingerprint of the whole schema. Merging deploys production, and an after-deploy job confirms the new version is live.
7. **Hand checks of real AI output.** CI uses the mock model, so translation quality and picture judgement are left for hand checks on production, from a short test pack; those for Sprints 7 and 8 are still to be done.

Running it

As a non-commercial project, it runs on free hosting plans within a budget of under £30 a month, and the only cost that varies is AI usage, which is capped. These gaps are accepted for a demo with invited users, and each one is on the backlog:

F-23	A failed check reports and emails, but on the current GitHub plan it cannot block a merge; the release process is the safeguard for now.
F-29	CI's Neon key is a repository secret, so a CI run on any branch could fetch production's connection string.
F-16a	No evaluation against the real models has been run yet, so AI quality rests on hand checks. The first is top of the backlog.
F-31	The Postgres driver carries a local patch for three connection bugs found on the day production moved to Neon, until a newer release is tested.

05

The agreement was adopted as decision 0021 on 2 November.

How AI was used to build it

Studio was built by Claude as an AI coding agent, working as lead engineer. I was the product manager: I owned the why, the what, and the sign-off. Claude also covered QA, platform and release engineering, and technical writing. Every sprint was refined, built test-first, reviewed, signed off, and closed, with anything unfinished returned to the backlog. From Sprint 7, sign-off came from a demo page built from CI's recordings, and an independent QA agent checked the tests first.

SPRINT	CLOSED	WHAT IT DELIVERED
1	25 Oct	Foundations: hosting, the agent contract, the spend cap, the decision trail, the hooks, and three design directions
2	25 Oct	The design system and brand kits, with two fictional demo brands and simulated customer data
3 · 4	26 Oct	The Strategy squad: audiences, the brief with evidence checks, market notes, personas, the test plan, and approvals
5	27 Oct	Creative I: copy, pictures, and ads composed in code, in a variant gallery
6	2 Nov	A move from Supabase to Neon, and Studio's own sign-in
7	3 Nov	Creative II: seven sizes, languages, and the launch pack, signed off from the first demo page
8	3 Nov (signed off)	Compliance checks, picture checks, uploaded-ad checks, and the synthetic pre-test panel

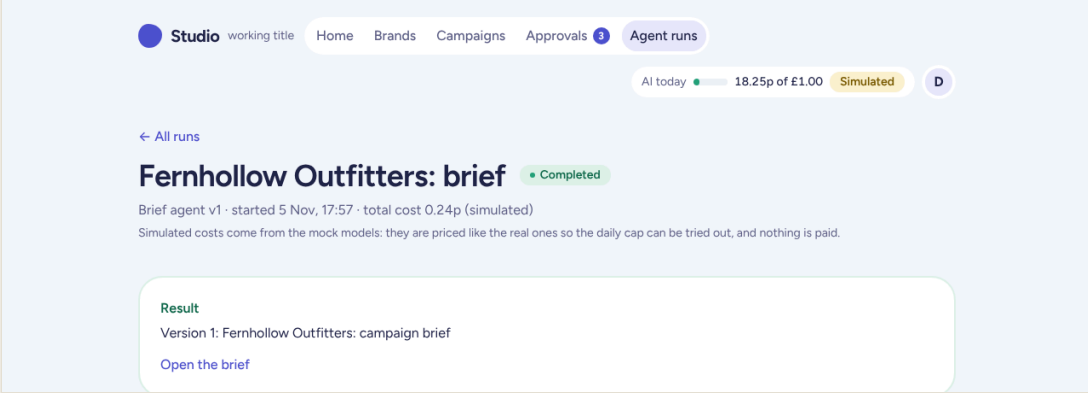
The roles inside the build

The repository gives Claude four specialist subagents, each with its own brief. A **test writer** turns each agreed case into a test before the code exists and confirms it fails for the right reason, with a cross-workspace test for every table and a prompt-injection test for every agent. An **agent engineer** builds product agents to the contract, on the cheapest model that passes its evaluations. A **UI builder** works only in the design system's tokens and checks each changed screen at 375 and 1440 pixels for AA contrast. A **reviewer** reads work it did not write, against the project rules, and ranks findings with a file, a line, and a fix; it does not edit code. A skill

scaffolds each new agent with its tests and evaluation file, and hooks run the type checks and lint after every edit to a TypeScript file, and the secret scan and tests before every commit.

What I did, and what changed because of it

Each refinement ended in questions for me, each with a recommendation. I accepted most of them, and changed some where the product needed it: invites became links I send myself, real rival names were allowed in market notes only and kept out of everything after them, and migrations are applied to Neon by Claude through the Neon connector, one file per transaction. The working agreement itself changed in Sprint 7, when the hand-run test packs I was working through turned out to repeat what a machine had already checked: from then on, I signed off from a page built from CI's recorded browser tests, and hand checks were kept for real AI output.



Every agent run has a decision trail: each step with its model, tokens, cost, time, and attempts. Here, with the mock model, the costs are simulated and labelled as such.

06

Eight project rules apply to every change, and the definition of done checks them.

Safety, by design and by test

RULE	WHERE IT IS ENFORCED
Honest by default	No invented metrics, testimonials, or claims. Simulated pictures are stamped in their pixels, translations are badged, panel scores say "Synthetic panel, not real customers", and findings say whether code, an estimate, or AI produced them.
Fictional brands only	Real names may appear only in market notes, with their sources. They are masked before the brief sees them, refused in briefs, copy, and image prompts, and an uploaded ad naming one fails.
People approve consequential actions	Approvals are pauses in the workflow. Creative starts only from a signed-off brief, replacements wait beside the original, creative and checking actions show their cost first, and review decisions are recorded with who and when.
Every run is traceable	Every step and every attempt is kept, with its raw reply, and the cost in the trail is read from the spending ledger.
Spend is capped	No model call bypasses the one gate that reserves its worst case under a lock.
Secrets stay out of the repository	A secret scan runs in the commit hooks and in CI; keys live only in Vercel and GitHub settings.
Tenancy from day one	Row-level security on every business table, reached only as the signed-in person, with a cross-workspace test for each.

RULE	WHERE IT IS ENFORCED
The image model never draws text	Prompts are checked for text requests, and a vision check on every picture redraws it once if it holds text, then drops it.

What testing found

The independent reviews found real problems before release. In Sprint 3, a citation tag "C37" let "37%" pass the number check, "45%" passed on a £45 price, "returns are guaranteed" slipped past the banned term "guarantee", and deleting a campaign erased its history; each was fixed with a test. Sprint 4's review found 19 issues, of which 18 were fixed and one accepted, and in Sprint 7 the independent QA agent found 12 problems with the tests themselves, including a browser test that could pass on a broken implementation. Sprint 8 was signed off with no failed cases, with its three hand checks still to be done on production.

Running the real models found what the mock could not. The Market agent took four rounds of fixes on its first production day: five searches read about 140,000 tokens for 30p, so they were cut to three; figures are now checked against the text of the cited page; possessives such as "YouGov's" were being flagged as company names; and a note that fails is now left out instead of failing the run. The Imagery agent needed three fixes, including a reply cut off at 1,500 tokens. Each production fix is the kind an evaluation against the real models should catch, which is why that evaluation is first on the backlog.

07 What it shows

The models in Studio do a small share of the work. Most of the product is the part around them: the kit every agent reads, the checks that run on everything they produce, the pauses where a person decides, and the trail that shows what happened. That is what would let a team trust a system to make 64 files it has not looked at one by one, and it is the part that transfers. The same shape fits any process where one decision becomes many checked variants: product listings across marketplaces, localised help content, regulated customer letters, or recruitment adverts across job boards.

Built with: Next.js 16 and React 19 on Vercel, Postgres on Neon with row-level security, Inngest, the Vercel AI SDK with Claude Sonnet 5.5 and Haiku 5.5 and Google's Nano Banana 2.1 image model, Satori for drawing ads, and Resend. All brands and figures in the app are fictional or simulated. The code is public at github.com/rorymcclelland/campaign-studio, and the screen-by-screen walkthrough is at mcclelland.info/work/studio.

Rory McClelland · product leader, AI products and governance
hello@mcclelland.info · mcclelland.info

Companion case studies:
RFI Respond · **Job Scout** · **How I build with AI**